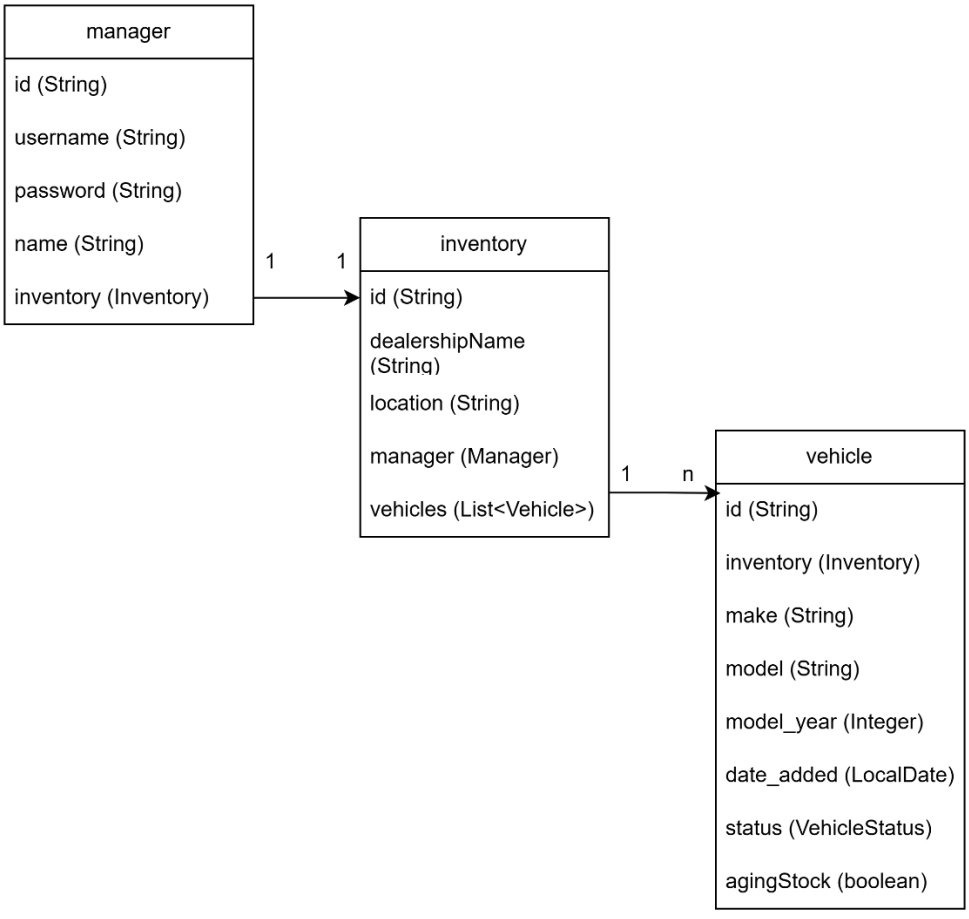
System Design Document for Supply Domain: The Intelligent Inventory Dashboard:

1) Introduction:

This document focuses on the system design and backend implementation of Scenario B: The Intelligent Inventory Dashboard in Supply domain of the technical coding challenge from Keyloop.

2) Entity Relationship Diagram:



*Note: Here I assume that each inventory is managed by one manager, and one dealership only has one inventory.

The inventory dashboard's main purpose is to help various dealership managers manage their inventories. Therefore, each manager can sign up/login to the dashboard and see their own dealership's inventory. Each manager has access to one inventory. Each inventory holds a list of vehicles. And each vehicle has important information that defines the vehicle like: make, model,

model year, age (how long the vehicle has been in inventory), and status (a proposed action that the manager has made for the vehicle, e.g., “Price Reduction Planned”).

3) Technical Solution:

3.1) Services:

Service	Methods	Input & Output	Purpose
UserRegistrationService	loadUserByUsername	Input: username Output: Spring Security’s UserDetails	Connect Spring Security to database to look up the username during login
ManagerDetailService	registerNewManager	Input: RegistrationRequest Output: ManagerDto	Create new manager with a new empty inventory in database after registration
	getManagerFromManagerId	Input: managerId (String) Output: GetManagerResponse	Returns manager’s details
InventoryDetailService	getInventoryFromManagerIdAndInventoryId	Input: managerId (String), inventoryId (String) Output: GetInventoryResponse	Returns inventory’s details
VehicleDetailService	addVehicleToInventory	Input: inventoryId (String), AddVehicleRequest Output: VehicleDto	Add a new vehicle to inventory
	getListOfVehiclesFromInventoryId	Input: inventoryId (String), VehicleFilterRequest Output: GetListOfVehiclesResponse	Returns a paginated and filtered list of vehicles in the inventory
	getVehicleFromInventoryIdAndVehicleId	Input: inventoryId (String), vehicleId (String) Output: GetVehicleResponse	Returns one vehicle detail
	changeVehicleStatus	Input: inventoryId (String), vehicleId (String), ChangeVehicleStatusRequest Output: VehicleDto	Change the status of a vehicle

3.2) DTOs:

Name	Fields	Purpose

AddVehicleRequest	- make (String, not blank) - model (String, not blank) - modelYear (String, not blank)	Requests to add a new vehicle to inventory
ChangeVehicleStatusRequest	- status (String, not blank)	Requests to change status of a vehicle
GetInventoryResponse	- inventoryDto (InventoryDto)	Returns inventory detail
GetListOfVehiclesResponse	- vehicles (Page<VehicleDto>)	Returns a paginated and filtered list of vehicles in the inventory
GetManagerResponse	- managerDto (ManagerDto)	Returns manager details
GetVehicleResponse	- vehicle (VehicleDto)	Returns vehicle details
InventoryDto	- id (String) - dealershipName (String) - location (String) - managerId (String) - managerName (String)	Inventory details
ManagerDto	- id (String) - username (String) - managerName (String) - inventoryId (String)	Manager details
RegistrationRequest	- username (String, not blank) - password (String, not blank) - name (String, not blank) - dealershipName (String, not blank) - location (String, not blank)	Requests to register a new manager
VehicleDto	- id (String) - inventoryId (String) - make (String) - model (String) - modelYear (String) - dateAdded (String) - status (String) - agingStock (boolean)	Vehicle details
VehicleFilterRequest	- page (int, default = 0) - size (int, default = 20) - make (String) - model (String) - maxAgeInDays (Integer)	Requests to filter a list of vehicles based on make/model/age

Enum	**Values**
VehicleStatus	- **AGING_REVIEW** ("Aging - Under Review")

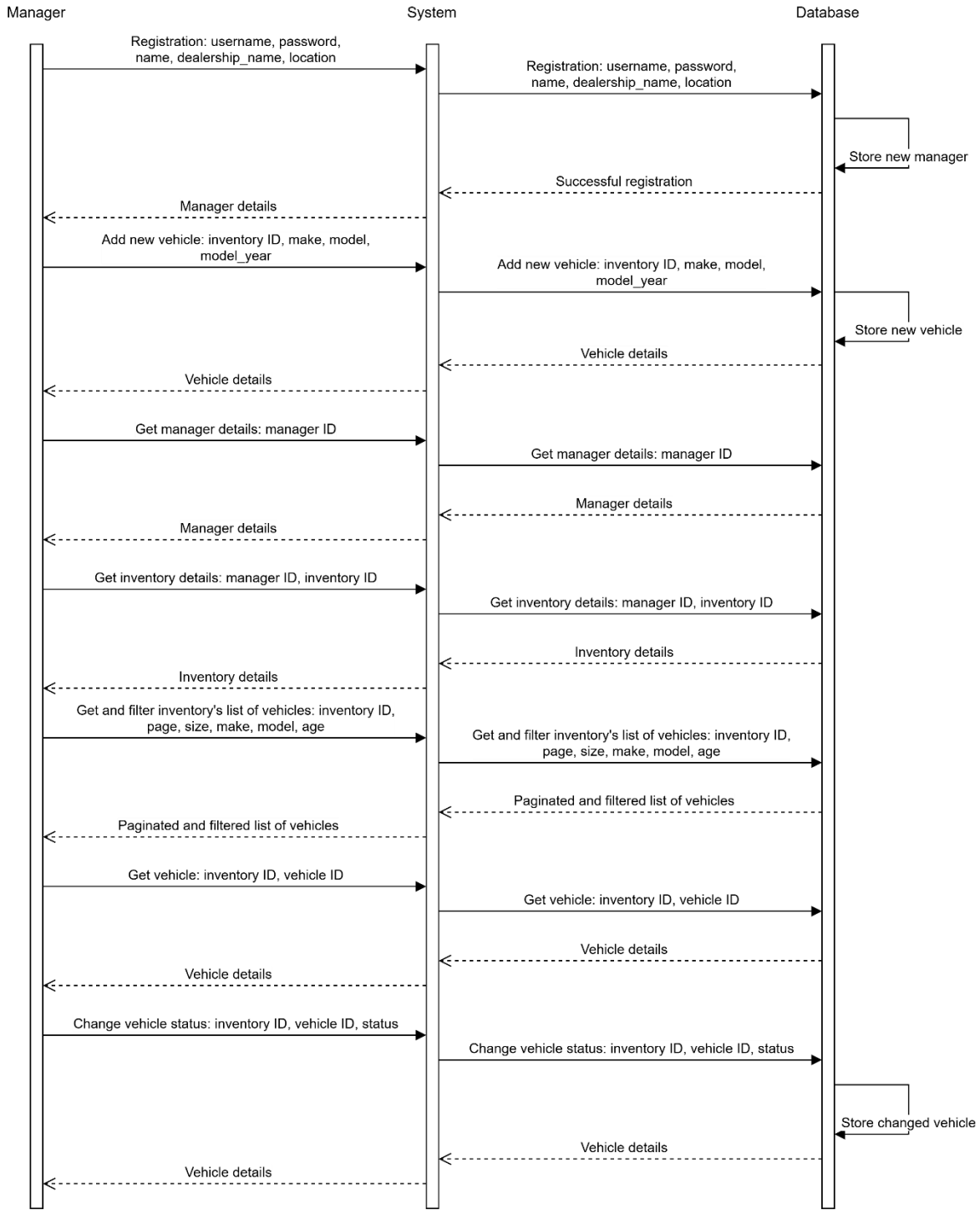
	- **AUCTION_READY** ("Scheduled for Auction") - **AVAILABLE** ("Available") - **DEALER_TRANSFER** ("Pending Dealer Transfer") - **HOLD** ("On Hold") - **PRICE_REDUCTION_PLANNED** ("Price Reduction Planned") - **SOLD** ("Sold")

3.3) Endpoints:

HTTP	**URI path**	**Input & Output**	**Purpose**
GET	/inventorydashboard/managers/{managerId} /inventories/{inventoryId}	Input: managerId, inventoryId Output: GetInventory Response	Get inventory from manager ID and inventory ID
GET	/inventorydashboard/managers/{managerId}	Input: managerId Output: GetManager Response	Get manager from manager ID
POST	/inventorydashboard/managers/{managerId}	Input: RegistrationR equest Output: ManagerDto	Register a new manager with a new inventory
POST	/inventorydashboard/inventories/{inventoryId} /vehicles/{vehicleId}/status	Input: inventoryId, vehicleId, ChangeVehicl eStatusRequ est Output: VehicleDto	Change the status of an aging vehicle in inventory
POST	/inventorydashboard/inventories/{inventoryId}/vehicles/ new	Input: inventoryId, AddVehicleR equest Output: VehicleDto	Add a new vehicle to inventory

GET	/inventorydashboard/inventories/{inventoryId}/vehicles	Input: inventoryId, VehicleFilterRequest Output: GetListOfVehiclesResponse	Get and filter a paginated list of vehicles from inventory
GET	/inventorydashboard/inventories/{inventoryId}/vehicles/{vehicleId}	Input: inventoryId, vehicleId Output: GetVehicleResponse	Get one vehicle from inventory ID and vehicle ID

3.4) All flows:



Registration: Manager inputs username, password, name, dealership name, location; system inserts new manager with a new inventory to database and returns manager details.

Add new vehicle: Manager inputs inventory ID with new vehicle's make, model, model year; system inserts new vehicle to database and returns vehicle details.

Get manager details: Manager inputs their ID, system queries and returns manager details from database.

Get inventory details: Manager inputs manager ID and inventory ID, system queries and returns manager's inventory details from database.

Get and filter list of vehicles: Manager inputs inventory ID, page and size for pagination, and filter request with a specific make/model/model year of the vehicles they want to see; system queries and returns paginated and filtered inventory's list of vehicles.

Get one vehicle: Manager inputs inventory ID and vehicle ID, system queries and returns the vehicle details.

Change vehicle status: Manager inputs inventory ID, vehicle ID, and new status for that vehicle; system save the new status of that vehicle in database and returns vehicle details.

3.5) Chosen technologies:

Java Spring Boot and PostgreSQL are chosen because:

- Spring Boot has autoconfiguration and provides strict type safety and architecture. It is commonly used in enterprise-level development, easy to maintain and scale. Spring Boot also offers Spring Security which is used for user registration/login in this dashboard system.
- PostgreSQL is a relational database that is suitable for data that has entities interconnected like in this system (every manager manages an inventory, every inventory has a list of vehicles, every vehicle has its own details).

JPA Repository and Hibernate are chosen to handle the persistence of data without having to handle the database management while the application is still rather small and simple. As the application scales up, migration scripts can be handled using Flyway.

Docker Compose is chosen for easily containerization of the application.

3.6) Strategy for Observability:

OpenTelemetry, the industry-standard and open-source observability framework, is integrated into the backend to collect traces. Jaeger is used to demonstrate these collected traces on a UI. Both of these frameworks will be built as Docker images.

4) The Use of GenAI in Design Phase:

For the design phase, I used Google Gemini to brainstorm a few vehicle statuses that are commonly used in vehicle inventory and assign them as VehicleStatus' values. Information about how to integrate registration/login function using Spring Security, and observability function using OpenTelemetry was also provided by Gemini. I also used the AI to understand more about the trade-offs for why many enterprise-level system uses Java Spring Boot and PostgreSQL in their backend development.